CS 5002 – Discrete Structures

Project 1 Part 1: Gradient Descent

Spring 2026

Wanjing Yang
NU ID: 002442139

Contents

1 Section 2.1 – Gradient Descent on Quadratic Functions **2**
1.1 Background . . . 2
1.2 Part 1 — Basic Test . . . 2
1.3 Part 2 — Varying x_0 . . . 3
1.4 Part 3 — Varying α . . . 3
1.5 Part 4 — Varying ε . . . 3

2 Section 2.2 — Non-Convex Function f_3 **4**
2.1 Background . . . 4
2.2 Part 2 — Multiple Starting Points . . . 5

3 Section 3 — Derivative Approximation **5**
3.1 Background . . . 5
3.2 Accuracy of the Approximation . . . 6
3.3 Gradient Descent with Approximate Derivative . . . 6

4 Section 4 — Gradient Descent for Functions of Two Variables **7**
4.1 Background . . . 7
4.2 Results on $f(x, y) = x^2 + y^2$ . . . 7

5 Conclusion **8**

1 Section 2.1 – Gradient Descent on Quadratic Functions

1.1 Background

We consider the two quadratic functions

$$f_1(x) = x^2 \qquad \text{and} \qquad f_2(x) = x^2 - 2x + 3. \tag{1}$$

Their exact derivatives are

$$f_1'(x) = 2x \qquad \text{and} \qquad f_2'(x) = 2x - 2. \tag{2}$$

A minimum occurs where $f'(x) = 0$. The gradient descent update rule steps opposite to the gradient:

$$\boxed{x_{k+1} = x_k - \alpha\, f'(x_k)} \tag{3}$$

where $\alpha > 0$ is the learning rate. Iteration stops when

$$|x_{k+1} - x_k| < \varepsilon \tag{4}$$

or the iteration count reaches `iter_max`.

1.2 Part 1 — Basic Test

Parameters: $x_0 = 3$, $\alpha = 0.1$, $\varepsilon = 0.001$, `iter_max`$= 1000$.

Table 1: Gradient descent results for f_1 and f_2 (basic test).

Function	x_0	x^*	$f(x^*)$	Iterations
f_1	3	0.003714	0.000014	29
f_2	3	1.003869	2.000015	27

Both functions converge close to their true minima ($x^* = 0$ for f_1 and $x^* = 1$ for f_2) with a small residual error controlled by ε.

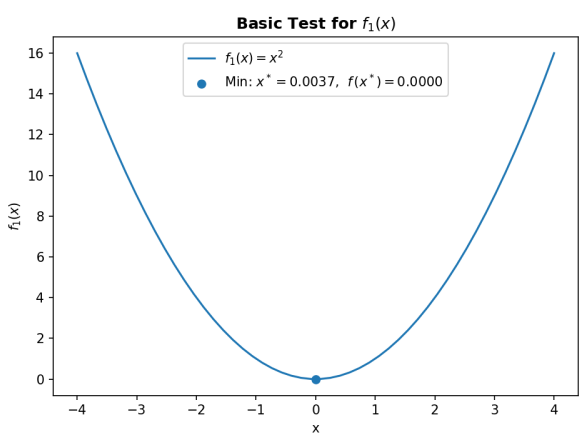

Figure 1: GD result on $f_1(x) = x^2$.

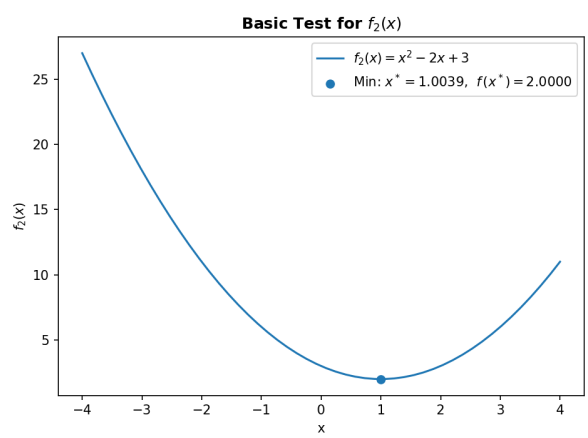

Figure 2: GD result on $f_2(x) = x^2 - 2x + 3$.

1.3 Part 2 — Varying x_0

Parameters: $x_0 \in \{3, -3\}$, $\alpha = 0.1$, $\varepsilon = 0.001$, `iter_max`= 1000.

Table 2: Effect of changing the starting point x_0.

Function	x_0	x^*	$f(x^*)$	Iterations
f_1	3	0.003714	0.000014	29
f_1	-3	-0.003714	0.000014	29
f_2	3	1.003869	2.000015	27
f_2	-3	0.996039	2.000016	30

Both functions are **convex** (bowl-shaped), so there is only one global minimum regardless of where we start. The starting point x_0 affects the number of iterations but not the final value found.

1.4 Part 3 — Varying α

Parameters: $x_0 = 3$, $\alpha \in \{1, 0.001, 0.0001\}$, $\varepsilon = 0.001$, `iter_max`= 1000.

Table 3: Effect of changing the learning rate α.

Function	α	x^*	Iterations	Observation
f_1	1	3.000000	1000	Diverges / oscillates
f_1	0.001	0.498984	895	Slow convergence
f_1	0.0001	2.999400	0	Step $< \varepsilon$ immediately
f_2	1	3.000000	1000	Diverges / oscillates
f_2	0.001	1.498455	693	Slow convergence
f_2	0.0001	2.999600	0	Step $< \varepsilon$ immediately

- $\alpha = 1$: too large — the update overshoots and the algorithm never converges within `iter_max`.
- $\alpha = 0.001$: valid but slow — many iterations are needed.
- $\alpha = 0.0001$: too small — the very first step is already smaller than ε, so the algorithm terminates at x_0 without making meaningful progress.

1.5 Part 4 — Varying ε

Parameters: $x_0 = 3$, $\alpha = 0.1$, $\varepsilon \in \{0.1, 0.01, 0.001\}$, `iter_max`= 1000.

Table 4: Effect of changing the tolerance ε.

Function	ε	x^*	Iterations
f_1	0.1	0.322123	9
f_1	0.01	0.034588	19
f_1	0.001	0.003714	29
f_2	0.1	1.335544	7
f_2	0.01	1.036029	17
f_2	0.001	1.003869	27

A looser tolerance (larger ε) terminates earlier with a less precise result. A tighter tolerance (smaller ε) requires more iterations but yields a more accurate minimum. There is a clear accuracy–cost trade-off.

2 Section 2.2 — Non-Convex Function f_3

2.1 Background

We consider the non-convex function

$$f_3(x) = \sin(x) + \cos\left(\sqrt{2}\,x\right), \quad 0 \le x \le 10, \tag{5}$$

with exact derivative

$$f_3'(x) = \cos(x) - \sqrt{2}\,\sin\left(\sqrt{2}\,x\right). \tag{6}$$

Unlike the quadratics in Section 2.1, f_3 has **multiple local minima** in $[0, 10]$.

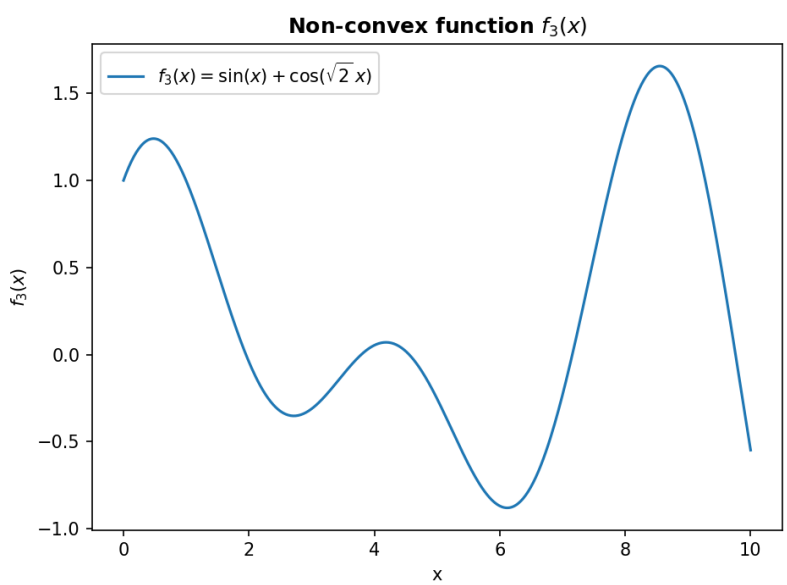

Figure 3: $f_3(x)$ over $[0, 10]$, showing multiple local minima.

2.2 Part 2 — Multiple Starting Points

Parameters: $x_0 \in \{1, 4, 5, 7\}$, $\alpha = 0.1$, $\varepsilon = 0.0001$, `iter_max`$= 1000$.

Table 5: GD on f_3 from four different starting points.

x_0	x^*	$f_3(x^*)$	Iterations
1	2.7156	-0.3524	65
4	2.7171	-0.3524	79
5	6.1180	-0.8805	49
7	6.1190	-0.8805	41

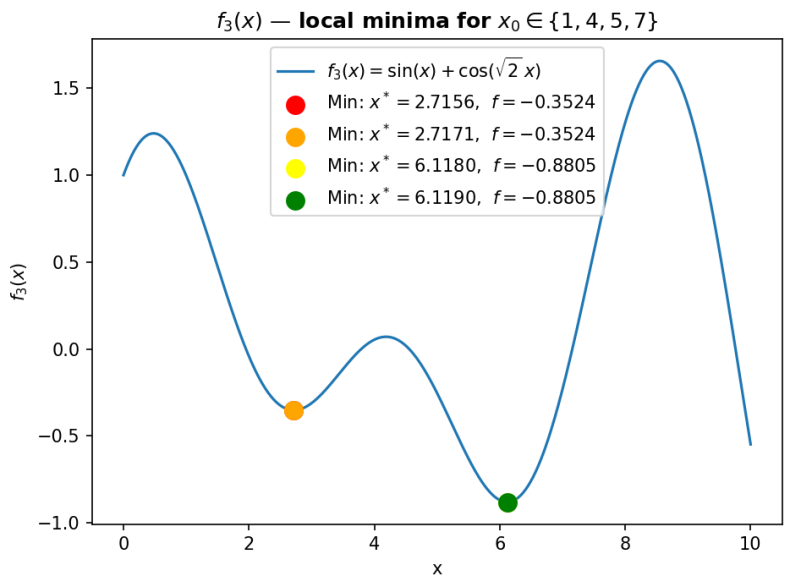

Figure 4: Local minima of f_3 found from four starting points.

Starting points $x_0 = 1$ and $x_0 = 4$ both converge to the same local minimum near $x^* \approx 2.72$ (valley A), while $x_0 = 5$ and $x_0 = 7$ both converge to a deeper local minimum near $x^* \approx 6.12$ (valley B). This demonstrates that for **non-convex** functions, the starting point x_0 determines *which* local minimum is found — there is no guarantee of reaching the global minimum.

3 Section 3 — Derivative Approximation

3.1 Background

When the exact derivative is unavailable, we approximate it using the **forward finite difference** formula, which follows directly from the limit definition of the derivative:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad h = 10^{-5}. \tag{7}$$

3.2 Accuracy of the Approximation

Table 6: Comparison of exact vs. approximated derivatives ($h = 10^{-5}$).

Function	x	Exact $f'(x)$	Approx $f'(x)$	Error
f_1	-5	-10.000000	-9.999990	1.00×10^{-5}
f_1	0	0.000000	0.000010	1.00×10^{-5}
f_1	2	4.000000	4.000010	1.00×10^{-5}
f_2	-5	-12.000000	-11.999990	1.00×10^{-5}
f_2	0	-2.000000	-1.999990	1.00×10^{-5}
f_2	2	2.000000	2.000010	1.00×10^{-5}

The approximation error is of order $h = 10^{-5}$, which is negligible for practical optimisation purposes.

3.3 Gradient Descent with Approximate Derivative

Using the approximated derivative inside gradient descent with $x_0 = 3$, $\alpha = 0.1$, $\varepsilon = 0.001$:

Table 7: GD with approximated derivative vs. exact derivative.

Function	x^* (approx)	x^* (exact)	Difference
f_1	0.003714	0.003714	~ 0
f_2	1.003869	1.003869	~ 0

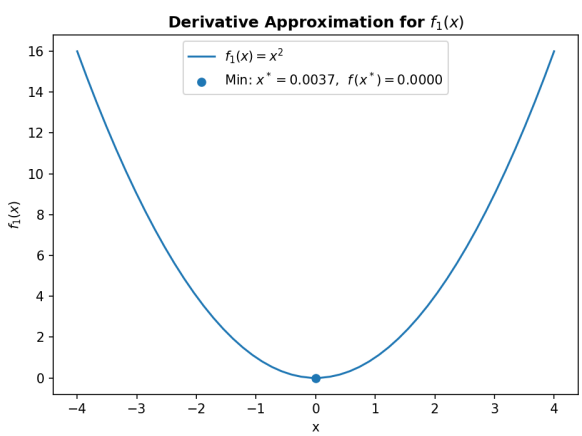

Figure 5: GD with approximated derivative on f_1.

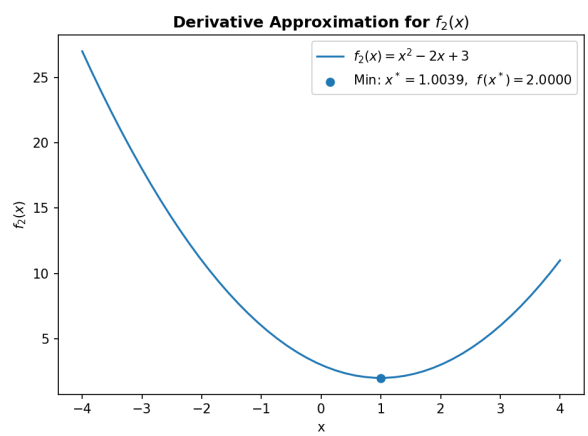

Figure 6: GD with approximated derivative on f_2.

The results are essentially identical to those obtained with the exact derivative. The $O(h)$ approximation error is far smaller than the stopping tolerance $\varepsilon = 0.001$ and therefore has no visible effect on the converged solution.

4 Section 4 — Gradient Descent for Functions of Two Variables

4.1 Background

We extend gradient descent to a function of two variables $f(x, y)$. Both variables are updated simultaneously at each iteration:

$$x_{k+1} = x_k - \alpha \frac{\partial f}{\partial x}(x_k, y_k), \qquad y_{k+1} = y_k - \alpha \frac{\partial f}{\partial y}(x_k, y_k). \tag{8}$$

Convergence is declared when *both* coordinate steps fall below ε:

$$|x_{k+1} - x_k| < \varepsilon \quad \text{and} \quad |y_{k+1} - y_k| < \varepsilon. \tag{9}$$

The partial derivatives are approximated using the same finite difference idea:

$$\frac{\partial f}{\partial x}(x_0, y_0) \approx \frac{f(x_0 + h,\, y_0) - f(x_0, y_0)}{h}. \tag{10}$$

4.2 Results on $f(x, y) = x^2 + y^2$

Parameters: $x_0 = y_0 = 3$, $\alpha = 0.1$, $\varepsilon = 0.001$, $h = 10^{-5}$, `iter_max`$= 1000$.

Table 8: 2D gradient descent result on $f(x, y) = x^2 + y^2$.

x^*	y^*	$f(x^*, y^*)$	Iterations
0.003709	0.003709	0.000028	29

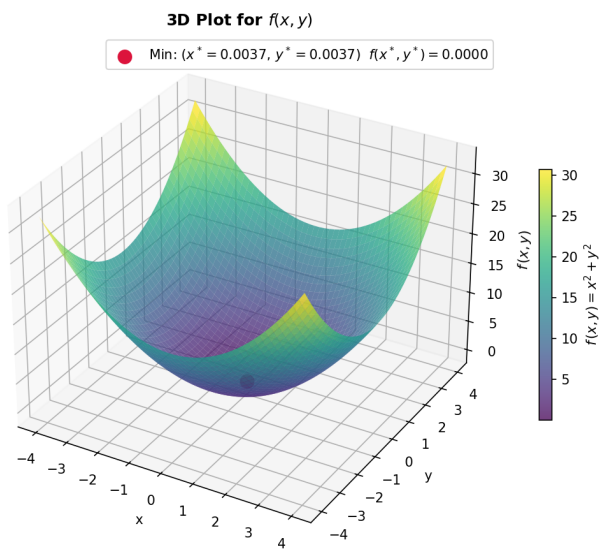

Figure 7: 3D surface of $f(x, y) = x^2 + y^2$ with the gradient descent minimum marked in red.

The algorithm converges to $(x^*, y^*) \approx (0.0037, 0.0037)$, very close to the true global minimum at $(0, 0)$. The result confirms that the 2D extension using approximate partial derivatives works correctly.

5 Conclusion

This project implemented and analyzed gradient descent across four settings. The key findings are:

- **Convex functions:** the starting point x_0 does not affect the final minimum, only the number of iterations.
- **Non-convex functions:** x_0 determines which local minimum is found; there is no guarantee of finding the global minimum.
- **Learning rate α:** too large causes oscillation/divergence; too small causes premature stopping or very slow convergence.
- **Tolerance ε:** controls the accuracy–cost trade-off; a tighter tolerance gives a more precise result at greater computational cost.
- **Derivative approximation:** forward finite differences with $h = 10^{-5}$ introduce an error of order 10^{-5}, negligible relative to typical stopping tolerances.
- **2D extension:** the framework generalises naturally to two variables using approximate partial derivatives, with identical hyperparameter trade-offs.